

Lightning Style Guide

Naming

Component names should be descriptive as to what the component does.

Lightning Applications and Lightning Events share the `AuraDefinitionBundle` Metadata type with Lightning Components. For organizing the tree view under `src/aura`, Lightning Applications and Events should be prefixed accordingly.

Lightning Apps should have "A_" prefixed to the front of the name.

There are two types of Lightning Events, Application and Component.

- Application Events should be prefixed "EA_".
- Component Events should be prefixed "EC_".

Components

Components should be treated just like any other class that you might create anywhere else.

1. They should have 1 responsibility.
2. They should be small.
3. They should be reusable.

When creating Lightning applications, you should be composing many small components into larger components, and eventually an application.

Naming

When naming your lightning component, follow the same guidelines as when naming an apex class. The name should describe exactly what the component's intent is.

Component

Lightning component markup should be treated just like your visualforce or html markup.

When given the opportunity, use the components supplied by salesforce instead of the native html tags (just like you would in visualforce). One thing that should be pointed out is that the beta components should never be used in any code that is going to production.

aura:if

When using the `aura:if` tag, use the following syntax (for performance reasons):

```
<aura:if isTrue="{!v.someValue}">
  True
  <aura:set attribute="else">
    False
```

```
        </aura:set>
    </aura:if>
```

Instead of:

```
<aura:if isTrue="{!v.someValue}">
    True
</aura:if>
<aura:if isTrue="{!!v.someValue}">
    False
</aura:if>
```

also with the `aura:if`, use operators instead of the 'gt', 'ne', etc.... as these are more readable.

aura:iteration

When using the `aura:iteration` tag, anything more than 1 line of markup should go into a new component, just like logic inside apex for loops.

Writing Declarative Components

A component is written declaratively when its dependencies are referenced directly within the component markup.

Example A below is preferable to Example B. When we write our component declaratively, we allow the framework to perform much of our overhead for us during the component life cycle. By creating and destroying components in our javascript, we are working outside of the component life cycle and we are making our component difficult to maintain.

Example A, Declarative

```
<!-- c:ParentComponent -->
<aura:component>
    <c:ChildComponent />
</aura:component>
```

Example B, Not Declarative

```
<!-- c:ParentComponent -->
<aura:component>
    {!v.body}
</aura:component>
```

```
// ParentComponent JS
({
  mountChild: function(component, event, helper) {
    $A.createComponent(
      "c:ChildComponent",
      {},
      function(childComponent, status, errorMessage) {
        var body = component.get("v.body");
        body.push(childComponent);
        component.set("v.body", body);
      });
  }
})
```

Designer

If there is any attribute that you think might possibly need changing in the future or you think would make sense for an end user or admin to change, put it in the designer.

When creating designer attributes, make sure to have a descriptive label (not descriptive for you, but descriptive to an end user), as well as a thorough description.

Keep in mind, if an attribute can be updated in the designer, you need to make sure you have fallbacks for if the field ends up empty.

Controller

Controller methods need to follow the same naming standards as all other methods. The only exception is the init function, which should be called `doInit`

All controller methods should also contain all 3 standard parameters (component, event, helper)

Lightning controller methods should follow the same guidelines as Apex controllers when it comes to what belongs in a controller. Controller methods should be kept skinny. The only thing that a controller method should be doing is taking in data from the view (the component) and sending data back to the view.

Here is an example of a controller w/ too much in it

```
deactivateOldAccounts: function(component, event, helper) {
  var accountsToDeactivate = component.get("oldAccounts");

  for (let i = 0; i < accountsToDeactivate.length; i++) {
    accountsToDeactivate[i].isActive = false;
  }

  var deactivateAccountsAction =
component.get("c.deactivateAccounts");
  action.setParams({ accountsToDeactivate: accountsToDeactivate });
  action.setCallback(this, function(response) {
    var state = response.getState();
```

```

        if (state === "SUCCESS") {
            // handle success
        } else if (state === "INCOMPLETE") {
            // handle incomplete response
        } else if (state === "ERROR") {
            // handle error
        }
    });
    $A.enqueueAction(deactivateAccountsAction);
}

```

This should be broken up into some separate helper methods

controller.js file

```

deactivateOldAccounts: function(component, event, helper) {
    var accountsToDeactivate = component.get("oldAccounts");

    helper.deactivateAccounts(helper, accountsToDeactivate);
}

```

helper.js file

```

deactivateAccounts: function(helper, accountsToDeactivate) {
    for (let i = 0; i < accountsToDeactivate.length; i++) {
        accountsToDeactivate[i].isActive = false;
    }

    helper.updateAccounts(accountsToDeactivate);
},

updateAccounts: function(accountsToUpdate) {
    var updateAccountsAction = component.get("c.updateAccounts");
    action.setParams({ accountsToUpdate: accountsToUpdate });
    action.setCallback(this, function(response) {
        var state = response.getState();
        if (state === "SUCCESS") {
            // handle success
        } else if (state === "INCOMPLETE") {
            // handle incomplete response
        } else if (state === "ERROR") {
            // handle error
        }
    });
    $A.enqueueAction(updateAccountsAction);
}

```

Here is an example of good sized controller methods. Note that the methods do not have any business logic in them, their only responsibility is to take data from the view and pass data to the view.

```
doInit: function(component, event, helper) {
    var activeAccounts = helper.getActiveAccounts(component);
    component.set("activeAccounts", activeAccounts);
},

deactivateAccount: function(component, event, helper) {
    var accountToDeactivate = component.get("selectedAccount");
    helper.deactivateAccount(component, accountToDeactivate);
}
```

Helper

Helper methods should follow the same size and responsibility rules as any other method in your code. These methods should have 1 responsibility, and you should start thinking about refactoring once your method hits around 5 lines of code (whitespace excluded).

Helpers do not require all 3 standard method params (component, event, helper), include just the ones you need.

If your helper method starts to get large, you can do 1 of 2 things:

- Create another helper method and call it with helper. syntax `helper.js file`

```
deactivateAccounts: function(helper, accountsToDeactivate) {
    for (let i = 0; i < accountsToDeactivate.length; i++) {
        accountsToDeactivate[i].isActive = false;
    }

    helper.updateAccounts(accountsToDeactivate);
},

updateAccounts: function(accountsToUpdate) {
    var updateAccountsAction = component.get("c.updateAccounts");
    action.setParams({ accountsToUpdate: accountsToUpdate });
    action.setCallback(this, function(response) {
        var state = response.getState();
        if (state === "SUCCESS") {
            // handle success
        } else if (state === "INCOMPLETE") {
            // handle incomplete response
        } else if (state === "ERROR") {
            // handle error
        }
    });
    $A.enqueueAction(updateAccountsAction);
}
```

When referencing another helper method, use `helper.whatever()` and not `this.whatever()`. Lightning sometimes gets confused with the `this` keyword, so just reference it with `helper`

- Pull the logic that is not directly related to lightning out into a static resource

static resource helper file

```
(function(window) {
    'use strict';

    window.accountCopier = {
        copyAccount: copyAccount
    };

    function copyAccount(accountToCopy) {
        var newAccount = {};

        // a bunch of logic needed to copy fields from
        accountToCopy to newAccount

        return newAccount;
    }
})(window || {});
```

Two things to note about using static resources: for LockerService to accept your code:

1. You have to be using strict mode
2. You have to put your code on the window object

account component file

```
<aura:component description="New account component">
    <ltng:require scripts="{!$Resource.accountCopier +
'/accountCopier.js'}" afterScriptsLoaded="{!c.doInit}" />

    <!-- The rest of your component -->
</aura:component>
```

helper.js file

```
createPrepopulatedAccount: function(accountToCopy) {
    var newAccount = accountCopier.copyAccount(accountToCop);
}
```

You get multiple benefits from option 2 above including:

- code reuse

- you can more easily unit test your shared business logic in your javascript (it is very difficult to do with javascript inside of a lightning component)
- your static resources will get cached, and even if you require them in multiple components, the lightning framework will only load it once

Style

The same CSS rules apply to css in a lightning component as a normal CSS file.

There are a few guidelines for where to put your styles though.

Global application or community level styles should go in a static resource and be required by the op level component.

If you start repeating styling rules in multiple components either push those rules up into a common parent component, or put them in a static resource.

The Lightning Design System uses a grid system, but it is actually flexbox behind the scenes, so you should have a general understanding of flexbox in order to save yourself from headaches in the future.

When building a Lightning Component, it's important to note that the first element beneath the `<aura:component>` is interpreted as `.THIS` in the component's CSS file. This can lead to confusing when attempting to target the top level element in CSS.

Examples: `component.css`

```
.THIS .container {  
    width: 100%;  
}
```

This will be interpreted to mean a hierarchy as shown in the following Lightning Component: `component.cmp`

```
<aura:component>  
    <div> <!-- .THIS -->  
        <div class="container">  
        </div>  
    </div>  
</aura:component>
```

On the other hand, to target the top level element with a specific class (E.g. different classes for component state) we must look at `.THIS` directly: `component.css`

```
.THIS.active {  
    background: green;  
}
```

This will be interpreted to mean a hierarchy such as `component.cmp`

```
<aura:component>
  <div class="active"> <!-- .THIS -->
  </div>
</aura:component>
```

Aura Docs

Aura Docs are a fantastic source of documentation for your components. You should use them.

When creating an aura doc, make sure to include an example of how to use your component, this is especially important for components that have multiple input attributes.

Your future replacement developers will thank you when they can run the aura doc report and have nice documentation of what each component does and how to use it.

Apex Controllers

Name your apex controllers with "AuraService" at the end so we can distinguish them from other controllers.

Apex controllers for lightning should be treated more like a REST endpoint than a traditional controller.

Try to keep your lightning apex controllers focused to the entity level instead of making them specific for a lightning component. Most of the time many components might end up getting similar data from the same apex controller.